

PE-EC703A — Embedded System

Semester VII • 3L:0T:0P • 3 Credits

Module 4 Exam Notes: Embedded Software Development & RTOS

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Assembly Language Programming (ALP) in Embedded Systems	2
1.1	Why ALP is Used in Embedded Systems	2
1.2	8051 ALP — Key Instructions	2
2	High-Level Language Programming — C for Embedded	3
2.1	Key C Concepts for Embedded Programming	3
2.2	Processor Directives in Embedded C	4
2.3	Functions and Macros in Embedded C	4
2.4	Embedded C++ Concept	5
3	Real-Time Operating System (RTOS)	5
3.1	OS Overview and Process Management	5
3.2	RTOS Task States	6
3.3	Task Scheduling Algorithms	6
3.4	Comparison of RTOS Scheduling Algorithms	7
3.5	Basic Design Rules Using RTOS	7

1. Assembly Language Programming (ALP) in Embedded Systems

Assembly Language Programming (ALP)

Assembly language is a low-level programming language that uses **mnemonics** (symbolic names for machine instructions) to directly control processor hardware. Each assembly instruction maps one-to-one to a machine code instruction. An **assembler** translates ALP source code to binary machine code.

1.1 Why ALP is Used in Embedded Systems

- **Maximum hardware control:** Direct access to all registers, flags, and ports.
- **Tight timing:** Exact instruction cycle count known — essential for time-critical routines (e.g., bit-bang communication, delay loops).
- **Minimum code size:** No compiler overhead; only absolutely necessary code is generated.
- **Boot code / startup:** Initial stack setup, clock configuration, and interrupt vector setup must be in assembly.
- **ISR optimization:** Critical ISRs are often hand-coded in assembly to minimize latency.

1.2 8051 ALP — Key Instructions

Instruction	Syntax / Example	Description
MOV	MOV A, #55H	Move immediate data to accumulator
ADD	ADD A, R0	Add R0 to Accumulator ($A = A + R0$)
SUBB	SUBB A, R1	Subtract R1 and Carry from A
SETB / CLR	SETB P1.0	Set/clear a specific bit
JMP / SJMP	SJMP LOOP	Unconditional short jump
JZ / JNZ	JZ DONE	Jump if accumulator is zero / not zero
DJNZ	DJNZ R2, LOOP	Decrement R2; jump if not zero (loop control)
LCALL / RET	LCALL DELAY	Long call subroutine / Return from subroutine
RETI	RETI	Return from interrupt (restores context)

8051 ALP Example: LED Toggle on Port 1

```

ORG 0000H          ; Program start address
MOV A, #0FFH       ; Load 0xFF into Accumulator (all bits high)
MAIN:
MOV P1, A          ; Send Accumulator to Port 1 (all LEDs ON)
ACALL DELAY        ; Call 1-second delay subroutine
CPL A              ; Complement Accumulator (0xFF -> 0x00)
SJMP MAIN          ; Repeat forever

DELAY:
MOV R0, #0FFH      ; Outer loop: 255 iterations
D1: MOV R1, #0FFH   ; Inner loop: 255 iterations
D2: DJNZ R1, D2      ; Decrement R1; loop until zero
    DJNZ R0, D1      ; Decrement R0; loop until zero
    RET             ; Return from subroutine
END

```

2. High-Level Language Programming — C for Embedded

C Language for Embedded Systems

C is the dominant high-level language for embedded systems programming. It provides a good balance between **hardware access** (via pointers, bitwise operations, volatile keyword) and **productivity** (structured code, functions, libraries). The C code is compiled by a cross-compiler for the target architecture.

2.1 Key C Concepts for Embedded Programming

Volatile Keyword

`volatile` tells the compiler that a variable's value can change at any time (e.g., by hardware or an ISR) and must not be cached in a register or optimized away:

```
volatile uint8_t flag = 0;    // Set by ISR, read by main loop
```

Bitwise Operations

Essential for setting, clearing, and testing individual bits in hardware registers:

Operation	C Code	Effect
Set a bit	<code>reg = (1 << n)</code>	Sets bit <i>n</i> without affecting others
Clear a bit	<code>reg &= ~(1 << n)</code>	Clears bit <i>n</i> without affecting others
Toggle a bit	<code>reg ^= (1 << n)</code>	Toggles bit <i>n</i>
Test a bit	<code>if (reg & (1 << n))</code>	Reads state of bit <i>n</i>

2.2 Processor Directives in Embedded C

Processor Directives

Processor directives (also called **preprocessor directives**) are instructions to the C preprocessor that run before actual compilation. They begin with # and control how the source code is processed.

Directive	Syntax	Purpose
#define	<code>#define LED_PIN 5</code>	Macro substitution; defines a constant or code snippet
#include	<code>#include "stm32f4xx.h"</code>	Includes a header file (device register definitions)
#ifdef / #endif	<code>#ifdef DEBUG ... #endif</code>	Conditional compilation (include code only for debug builds)
#pragma	<code>#pragma once</code>	Compiler-specific directive (prevent double-inclusion)
#ifndef	<code>#ifndef _HEADER_H</code>	Include guard to prevent multiple inclusions

2.3 Functions and Macros in Embedded C

- **Functions:** Reusable code blocks. Function calls add overhead (stack push/pop of parameters and return address). For embedded systems, frequently called short functions can be declared `inline` to avoid this overhead.
- **Macros:** Defined with `#define`. Expanded inline by the preprocessor — zero runtime overhead. Used for hardware register bit manipulation, delays, and port pin aliases.

C Macro vs. Inline Function Example

```
/* Macro -- no type safety, zero overhead */
#define SET_BIT(reg, bit) ((reg) |= (1U << (bit)))
#define CLEAR_BIT(reg, bit) ((reg) &= ~(1U << (bit)))

/* Inline function -- type-safe, zero overhead (compiler expands inline) */
static inline void gpio_set(volatile uint32_t *reg, uint8_t bit) {
    *reg |= (1U << bit);
}

/* Usage */
SET_BIT(GPIOA->ODR, 5);    /* Macro: sets PA5 */
gpio_set(&GPIOB->ODR, 3);  /* Inline function: sets PB3 */
```

2.4 Embedded C++ Concept

Embedded C++ (EC++)

Embedded C++ is a subset of C++ designed for resource-constrained embedded targets. It uses C++ features that add **zero or minimal overhead** while providing better code organization and type safety compared to plain C.

C++ features commonly used in embedded:

- **Classes and Objects:** Encapsulate peripheral drivers (e.g., a UART class with `send()` and `receive()` methods).
- **Namespaces:** Prevent name collisions between libraries (e.g., `hal::uart::init()`).
- **Templates:** Generic, type-safe containers and algorithms (e.g., ring buffer template).
- **constexpr:** Compile-time constant evaluation reduces runtime overhead.
- **RAII (Resource Acquisition Is Initialization):** Automatic resource management (locks released when object goes out of scope — critical for RTOS mutexes).

C++ features AVOIDED in embedded (high overhead):

- Exceptions (`try/catch`) — adds large overhead and unpredictable timing.
- Runtime Type Information (RTTI) / `dynamic_cast` — requires type tables at runtime.
- `new/delete` (dynamic heap allocation) — causes fragmentation and non-deterministic timing.

3. Real-Time Operating System (RTOS)

RTOS — Real-Time Operating System

An **RTOS** is an operating system designed for **deterministic, time-bound execution** of tasks. It provides task scheduling, inter-task communication (queues, semaphores, mutexes), memory management, and timer services. Unlike general-purpose OS, an RTOS guarantees task completion within defined time limits.

3.1 OS Overview and Process Management

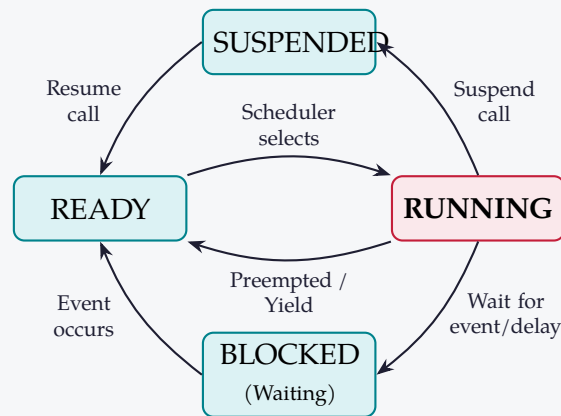
RTOS Services

- **Task (Thread) Management:** Create, delete, suspend, resume tasks with different priorities.
- **Scheduling:** Algorithm that decides which task runs on the CPU at any given time.
- **Inter-Task Communication (IPC):** Queues (FIFO data), semaphores (signaling), mutexes (shared resource protection).
- **Memory Management:** Static or dynamic allocation of memory blocks to tasks.

- **Timers:** Software timers for periodic callbacks without a dedicated hardware timer.
- **Examples:** FreeRTOS, Zephyr, RTEMS, VxWorks, uC/OS-II, ThreadX.

3.2 RTOS Task States

RTOS Task State Diagram



3.3 Task Scheduling Algorithms

1. Priority-Based Preemptive Scheduling

Priority-Based Preemptive Scheduling

Each task is assigned a **priority** (numeric value). The scheduler always runs the highest-priority task that is in the READY state. If a higher-priority task becomes ready while a lower-priority task runs, the lower-priority task is **immediately preempted** (stopped) and the higher-priority task takes the CPU.

- **Deterministic:** Highest-priority tasks always respond within bounded time.
- **Starvation risk:** Very low-priority tasks may never run if higher-priority tasks are always ready.
- **Priority Inversion:** A high-priority task is blocked because a low-priority task holds a shared resource. Solved by **Priority Inheritance Protocol**.
- **Used in:** FreeRTOS, VxWorks, RTEMS, most industrial RTOS.

2. Cyclic Scheduling (Time-Division Multiplexing)

Cyclic Scheduling

Tasks are arranged in a **fixed cyclic sequence** (also called a **static schedule**). A major cycle (hypercycle) is divided into **time slots**, and each task runs during its assigned slot. No dynamic priority decisions — the schedule is fixed at design time.

- **Predictable:** Exact execution times are known at compile time.

- **Simple:** No complex scheduler overhead.
- **Inflexible:** Cannot respond to dynamic events; hard to add new tasks.
- **Used in:** Safety-critical aerospace and automotive systems (ARINC 653, AUTOSAR).

3. Round-Robin Scheduling

Round-Robin Scheduling

All tasks of **equal priority** are given equal CPU time **time slices** (quantum, e.g., 10 ms). The scheduler cycles through all ready tasks in a circular queue. When a task's time slice expires, it is preempted and the next task in the queue runs.

- **Fair:** All equal-priority tasks share CPU time equally.
- **No starvation** for equal-priority tasks.
- **Response time:** Depends on the number of tasks and the time quantum (worst case: $(n - 1) \times Q$).
- **Used in:** Linux time-sharing, RTOS task groups at same priority.

3.4 Comparison of RTOS Scheduling Algorithms

Feature	Priority-Based	Cyclic	Round-Robin
Flexibility	High	Low	Medium
Determinism	High	Very High	Medium
Fairness	Low (priority skew)	Fixed	High (equal priority)
Overhead	Medium	Very Low	Low
Starvation	Possible	No	No (equal priority)
Best For	General RTOS	Safety-critical	Time-sharing tasks

3.5 Basic Design Rules Using RTOS

1. **Assign priorities carefully:** Critical, fast-response tasks (e.g., motor control) get the highest priority. Background tasks (e.g., logging) get the lowest.
2. **Use semaphores/mutexes** to protect shared resources (global variables, hardware peripherals) from concurrent access by multiple tasks.
3. **Avoid blocking calls in high-priority tasks:** Use non-blocking APIs or timeouts.
4. **Minimize stack usage:** Each task has its own stack. Over-allocating wastes RAM; under-allocating causes stack overflow (hard to debug).
5. **Keep ISRs minimal:** ISR sets a flag or sends to a queue; task processes the data.
6. **Use RTOS trace tools:** Tools like FreeRTOS+Trace or Segger SystemView to observe task switching, timing, and CPU usage.

Quick Revision — Module 4

Embedded Software Development & RTOS | PE-EC703A

Key Formulas at a Glance

8051 Delay Loop Cycles	$N_{cycles} = (\text{count}_{outer}) \times (\text{count}_{inner}) \times 5$ (approx., per DJNZ)
RTOS Response Time (RR)	$t_{response} \leq (n - 1) \times Q$ (n = tasks at same priority, Q = time quantum)
CPU Utilization Check	$U = \sum_{i=1}^n \frac{C_i}{T_i} \leq 1$ (C_i = exec. time, T_i = period of task i)

One-Line Definitions

- **ALP:** Assembly Language Programming — low-level mnemonic-based programming for direct hardware control.
- **Assembler:** Translates assembly (mnemonic) code to binary machine code.
- **Cross-compiler:** Compiler on a host PC generating code for a different target (e.g., ARM GCC).
- **volatile:** C keyword that prevents compiler from caching a variable that can change externally (e.g., ISR flag).
- **Preprocessor Directive:** Instructions beginning with # that control compilation before the compiler runs.
- **Inline function:** C++ function expanded at call site by the compiler, eliminating function call overhead.
- **RTOS:** Operating system guaranteeing deterministic task scheduling with hard timing deadlines.
- **Task (Thread):** An independent unit of execution managed by the RTOS scheduler.
- **Preemption:** RTOS forcibly stopping a running task to run a higher-priority task.
- **Semaphore:** RTOS signaling mechanism for synchronization between tasks or between ISR and tasks.
- **Mutex:** Mutual exclusion object; protects shared resources from concurrent access (includes priority inheritance).
- **Priority Inversion:** A high-priority task blocked by a low-priority task holding a needed resource.
- **Round-Robin:** Scheduling algorithm giving equal time slices to all tasks at the same priority level.

Frequently Asked / Viva Questions**Q1. What is the difference between assembly language and C for embedded programming?**

ALP gives maximum hardware control, exact cycle-count timing, and minimum code size but is processor-specific and harder to maintain. C provides portability, structured code, and faster development with slightly larger code. Both are commonly mixed: C for application logic, ALP for startup and timing-critical routines.

Q2. What is the purpose of the `volatile` keyword in embedded C?

It tells the compiler that a variable can change unexpectedly (by an ISR, DMA, or hardware register) and must not be optimized or cached. Without `volatile`, the compiler may read the variable once and reuse the cached value, missing updates.

Q3. What are preprocessor directives? Give two examples.

Instructions beginning with `#` that are processed before compilation. Examples: `#define LED 5` (macro constant), `#include "stm32.h"` (file inclusion), `#ifdef DEBUG` (conditional compilation).

Q4. What is an RTOS and how does it differ from a bare-metal embedded system?

An RTOS provides multi-tasking, scheduling, and synchronization services. In bare-metal, a single `while(1)` loop and ISRs handle everything sequentially — no time guarantees between events. RTOS ensures tasks run within defined deadlines and can run concurrently.

Q5. Explain priority-based preemptive scheduling.

Each task has a priority number. The RTOS scheduler always runs the highest-priority READY task. If a higher-priority task becomes ready (e.g., unblocked by an ISR posting to its queue), the currently running lower-priority task is immediately preempted and the higher-priority task takes the CPU.

Q6. What is priority inversion and how is it solved?

Priority inversion: a high-priority task (H) is blocked waiting for a resource held by a low-priority task (L), while a medium-priority task (M) runs and prevents L from completing. **Solution:** Priority Inheritance — temporarily raise L's priority to H's level until it releases the resource.

Q7. Compare cyclic scheduling and round-robin scheduling.

Cyclic scheduling uses a fixed time-division slot sequence (fully deterministic, best for safety-critical systems, inflexible). Round-robin dynamically gives equal time quantum to all equal-priority tasks (fair, flexible, slight unpredictability in deadline, good for general tasks).

Q8. What is the difference between a semaphore and a mutex?

A **semaphore** is a signaling mechanism (binary or counting) primarily used for task synchronization and ISR-to-task signaling. A **mutex** (Mutual Exclusion) is specifically for protecting shared resources from concurrent access and includes priority inheritance to prevent priority inversion.

Frequently Asked / Viva Questions (contd.)

Q9. What is the role of the RTOS kernel?

The RTOS kernel manages task creation, deletion, scheduling (context switching), inter-task communication (queues, semaphores), timers, and memory allocation. It runs the context switch code that saves and restores task registers during task switching.

Q10. What are the advantages of C++ over C in embedded programming?

C++ provides encapsulation (classes for peripheral drivers), type safety (templates), compile-time computation (constexpr), and RAII for automatic resource management. These improve code organization and reliability with zero runtime cost if high-overhead features (exceptions, RTTI, heap) are avoided.

Q11. What is a time quantum in round-robin scheduling?

The time quantum (slice) is the maximum CPU time a task runs before being preempted and the next task getting a turn. Shorter quantum gives better responsiveness but more context-switch overhead. Typical values: 1–20 ms.

Q12. What is the DJNZ instruction in 8051 assembly and why is it used?

DJNZ Rn, label — Decrement Register *n* and Jump if Not Zero. It is the standard loop instruction: `MOV R0, #100; LOOP: DJNZ R0, LOOP` creates a 100-iteration delay loop with a 2-machine-cycle instruction — very efficient for timing loops.

RTOS Scheduling: Priority-Based vs. Round-Robin Timeline

